

XRPL Validator — Daily Engineering Report • 2026-08-08

Compiled retrospectively on 2026-09-23 from the git history. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the XRP Ledger itself (amendments) and XRPL research.

Summary

- **11 commits** (xrpl-validator 11)

Engineering changes

- **fix(xrpl-ledger): single-path AMM takes maxOffer, clamped through the pool BOTH ways** (3180e04 , xrpl-validator) Closes #106137477 CD634127CD8B (86/87 -> 87/87) and wires in the `max_offer` built in a171405. The clamp that a171405 named as "the missing half" turned out to be TWO-directional, which is what defeated that attempt.
- **fix(xrpl-ledger): limitOut sizes a pass TO the limit — #106134431 closed 122/122** (3033135 , xrpl-validator) Completes the AMM bridge slice-sizing family. Every divergence from the 2026-08-07 fresh batch now matches mainnet.
- **fix(xrpl-ledger): destination checks across every transactor that has them** (b569fa9 , xrpl-validator) 5626357 fixed destination-check ORDERING in payment.rs and I called the class closed. It was not: the very next fresh batch found the same class wrong in a second Payment branch AND in a transactor I had not looked at. So this time I grepped every transactor that emits the code -- rippled has `tecDST_TAG_NEEDED` in SEVEN, we had it in TWO.
- **fix(xrpl-ledger): the reverse pass must size for the pass the forward one runs** (89bed1e , xrpl-validator) #106143011 5D327F343A54 -- Payment, ours `tecPATH_PARTIAL` vs net `tesSUCCESS`, 1 mutation against 33. Circular `tfPartialPayment`, `SendMax` 100 SPY into 18.00944028251864 RLUSD via SPY->XRP->RLUSD, `DeliverMin` 17.91939308110604.
- **fix(xrpl-ledger): tfNoRippleDirect forbids the strand we fall back to** (7d785ec , xrpl-validator) #106146562 C56D61917E4B -- Payment, ours `tesSUCCESS` with 4 mutations vs net `tecPATH_DRY` with 1. Three EXTRA nodes, none missing.
- **fix(xrpl-ledger): size the reverse-pass grant to the requirement it measures** (125be81 , xrpl-validator) #106148286 4EFC975484E1 -- Payment, ours `tecPATH_PARTIAL` vs net `tesSUCCESS`, 1 mutation against 9. Circular `tfPartialPayment|tfNoRippleDirect`, `SendMax` 0.02117670765212899 mXRP for 2.302493172774841 CORE over a FOUR-leg chain of pools, mXRP -> BitX -> BTC -> CORE. One strand, so 89bed1e is inert here: same symptom as #106143011, different regime.
- **fix(xrpl-ledger): a bridge leg-B maker cannot deliver more than it holds** (5dc7cb7 , xrpl-validator) #106146362 75511674AD58 -- `tfSell OfferCreate`, 1000 XAH for 1 RLUSD, autobridged XAH->XRP->RLUSD. Both engines `tesSUCCESS`; ours 11 mutations against mainnet's 16, 5 missing, none extra.
- **feat(xrpl-node): DX_VALCHECK — compare the VALUES we wrote, not just the keys** (9bc95c8 , xrpl-validator) The differential probe's verdict is a mutation-set comparison, and a mutation set is KEYS. Every value

a transaction writes is invisible to it.

- **fix(xrpl-ledger): a rested residual keeps its ORIGINAL quality in its AMOUNTS** (850da82 , xrpl-validator)
The book page for a partially crossed offer has always been built from the offer as REQUESTED (tp0 / tg0) --rippled keeps a partial fill at its original price. The AMOUNTS written into that offer did not follow: they were TakerGets - consumed , i.e. reduced by the currency actually handed over.
- **fix(xrpl-ledger): NFTokenCreateOffer never charged for the object it creates** (a56ccbd , xrpl-validator)
#106154459 A55AD53EEFA2 -- ours tesSUCCESS with 5 mutations vs mainnet tecINSUFFICIENT_RESERVE with 1. All five EXTRA: the NFTokenOffer plus both directory pages.
- **feat(xrpl-ledger): one strand per named path, and stop gating hops with the payment's limit** (868cf89 , xrpl-validator) #106156904 341030105165 -- ours tecPATH_DRY with 1 mutation vs mainnet tesSUCCESS with 7. Circular tfPartialPayment|tfLimitQuality, SendMax 7.203545999999999 USDT for 0.095 SOL, and Paths carries SIX paths.

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **Threshold Crypto-Conditions for Checks** (payments) Extend CheckCreate with an optional sfCondition SField using CryptoConditionsSuite's expanded condition types (THRESHOLD-SHA-256, PREFIX-SHA-256, PREI
- **Credential-Gated Guardian Account Recovery** (accounts) Add a RecoverList ledger object (ltRECOVER_LIST, owner-reserved, keylet::recover(account)) created via RecoverListSet, holding sfGuardians (AccountID
- **Reserve-Gated Wrapped-Asset Minting** (audit) Add lsfRequireReserveProof, sfReserveAttestationID (a keylet reference to an on-ledger reserve-attestation object), sfMinReserveBps, and sfMaxReserveA

Appendix — commits

Time	Repo	Commit	Subject
00:14	xrpl-validator	3180e04	fix(xrpl-ledger): single-path AMM takes maxOffer, clamped through the pool BOTH ways
01:13	xrpl-validator	3033135	fix(xrpl-ledger): limitOut sizes a pass TO the limit — #106134431 closed 122/122
03:59	xrpl-validator	b569fa9	fix(xrpl-ledger): destination checks across every transactor that has them
06:04	xrpl-validator	89bed1e	fix(xrpl-ledger): the reverse pass must size for the pass the forward one runs
06:49	xrpl-validator	7d785ec	fix(xrpl-ledger): tfNoRippleDirect forbids the strand we fall back to
07:36	xrpl-validator	125be81	fix(xrpl-ledger): size the reverse-pass grant to the requirement it measures
11:34	xrpl-validator	5dc7cb7	fix(xrpl-ledger): a bridge leg-B maker cannot deliver more than it holds
14:03	xrpl-validator	9bc95c8	feat(xrpl-node): DX_VALCHECK — compare the VALUES we wrote, not just the keys
18:17	xrpl-validator	850da82	fix(xrpl-ledger): a rested residual keeps its ORIGINAL quality in its AMOUNTS
19:30	xrpl-validator	a56ccbd	fix(xrpl-ledger): NFTokenCreateOffer never charged for the object it creates
21:36	xrpl-validator	868cf89	feat(xrpl-ledger): one strand per named path, and stop gating hops with the payment's limit